
What the Final Patch Hides: Event-Level Regression Measurement for Coding-Agent Harnesses

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Coding agents are evaluated on the patch they leave behind. We show that this
2 final state hides most of what happens during a run. We built an instrument that
3 records every mutating action of an agent as a git commit on a hidden reference,
4 replays each benchmark instance’s held-out tests at every recorded state inside
5 the benchmark’s own container image, and attributes each detected regression to
6 the harness check that could have caught it. Applied to SWE-bench Verified, the
7 instrument recorded 184 regression events across the runs that produced any; the
8 final patch exposed one of them. Regression frequency depended on the model
9 stack rather than the benchmark: on the same 40 instances under the same harness,
10 one frontier stack produced no events in 40 runs and another produced 140 events in
11 11 incidents (Fisher $p = 0.010$). A pre-registered comparison of recovery policies,
12 analysed with code committed before unblinding, found that rollback to a verified
13 checkpoint left contamination in one final tree where no recovery left it in nine
14 (paired sign test, $p = 0.022$), but rollback also resolved fewer tasks, because the
15 verifier that triggers it rejected patches the official grader accepted. Replacing that
16 verifier with the repository’s own tests removed the tradeoff: 65% resolved with no
17 contaminated final tree (McNemar $p = 0.012$ against plain rollback). We release
18 the instrument, its validation protocol, and a catalogue of 28 measurement defects
19 encountered while building it, 23 of which produced plausible numbers rather than
20 errors.

21 1 Introduction

22 Benchmarks for coding agents grade the final patch. SWE-bench [1] and its Verified subset [2] run
23 the repository’s tests against the patch an agent submits and report whether the target tests now pass
24 and the previously passing tests still pass. Work on regressions introduced by agents follows the same
25 convention: it examines the submitted patch and counts the previously passing tests it breaks [8].

26 This convention has a blind spot. A regression that an agent introduces and later repairs leaves no
27 trace in the final patch, and any harness with a recovery mechanism produces that pattern by design.
28 Whether it matters depends on two unanswered empirical questions: how often regressions occur
29 during a run, and how much of that activity survives to the final state.

30 We answer both by measuring the timeline rather than the endpoint. Our contributions are:

31 1. **An instrument for event-level regression measurement.** Every mutating tool call is recorded as
32 a commit on a git reference the agent cannot see. After the run, the instance’s held-out passing tests
33 are replayed at every recorded state inside the benchmark’s pinned container. Detection is attributed
34 by test coverage rather than by co-occurrence in time. The instrument’s validity is established by

positive and negative controls, a baseline liveness gate, a golden check that drives the benchmark’s own gold patch through the agent’s execution path, and a re-scoring control that re-measures archived runs under a changed instrument with no model calls (Section 3). 2. **Measurements on SWE-bench Verified.** Across the runs that produced any regression, the timeline recorded 184 events and the final state exposed one (Section 5.2). Event frequency was a property of the model stack, not the benchmark: identical instances and harness gave 0 events under one frontier stack and 140 under another (Section 5.3). 3. **A pre-registered comparison of recovery policies, and a fix.** With the analysis committed before unblinding, rollback to a verified checkpoint left fewer contaminated final trees than no recovery ($p = 0.022$) but resolved fewer tasks; we trace the loss to verifier precision, and a regression-gated verifier built from the instrument’s own oracle removes it, reaching 65% resolve with no contaminated tree (Sections 5.4 to 5.6).

All measurements are exploratory and were made on a development slice of the benchmark that is excluded from any confirmatory frame. No claim is made about the population of instances beyond that slice.

2 Background and related work

Benchmarks and their validity. SWE-bench [1] grades a patch by tests that must go from failing to passing and tests that must keep passing; Verified [2] is the human-screened 500-instance subset. Concerns about the static subset have accumulated: contamination and memorisation [6], leaked solutions and flawed tests [6, 7], and rolling [3, 5] or private [4] alternatives in response. UTBoost [7] showed the held-out tests are often insufficient, changing 24% of Verified leaderboard verdicts. TDAD [8] measured regressions in submitted patches on Verified and found them in about 30% of resolved instances; we measure the same quantity on the timeline, and Section 5.2 compares the two on the same runs.

Agent scaffolds and recovery. SWE-agent [9], OpenHands [10], Agentless [11], and mini-swe-agent [12] span the scaffold design space, on the action formalisms of CodeAct [13] and ReAct [14]. Within-episode recovery has been studied as self-reflection [15, 16], progress-gated recovery [17], checkpoint repair [18], and provenance-based rollback [19]; operating-system framings [20] treat checkpointing as a shared primitive. Process reward models and rubric verifiers [24, 25] approach the verifier-precision problem of Section 5.5 from the training side.

Evaluation infrastructure. Trajectory-level diagnostics [22], holistic leaderboards [21], and long-horizon benchmarks [23] argue that resolve rate alone is uninformative. Our instrument is a validated trajectory-level measure of one quantity.

Automated program repair. The regression problem is the plausible-versus-correct patch problem of automated program repair [26, 28, 29]; a study of two million patches [27] found regression tests to be the main defence against overfitting patches. Our results concern regressions the agent’s own process repairs before the patch is ever tested.

3 Instrument

Observational timeline. The harness wraps every mutating tool call. After each call, the working tree is committed to a git reference outside the agent’s view, using a private index so that the agent’s own git status and git diff are unchanged. Rollbacks and the end of the run are recorded as observations too. Because observations are commits, an archived run can be re-measured later without re-running the agent.

Exhaustive replay. For each observation, the instance’s passing-test set is run inside the benchmark’s pinned container against that observation’s tree. A regression event is a test that passes at one observation and fails at a later one. We replay every observation rather than bisecting between the start and the end, because bisection assumes the verdict sequence is monotone, and a recovery policy violates that assumption by construction. Infrastructure failures during replay (a container that will not start, a patch that will not apply, a timeout, a log with no results) are recorded as missing observations, never as failures. Baseline-dead tests, which fail at the first observation, are excluded from event counting.

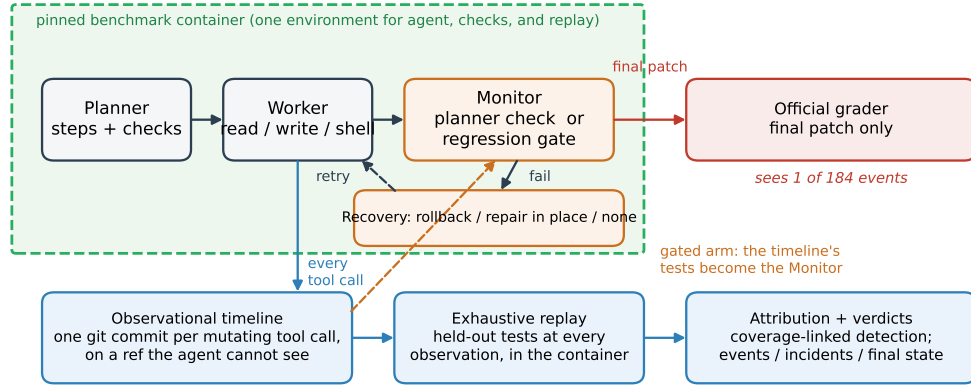


Figure 1: The harness and the instrument. The planner, worker, monitor, and recovery policy run inside the benchmark’s pinned container; every mutating tool call is committed to a hidden git reference; held-out tests are replayed at every observation and detections are attributed by coverage. In the gated arm (dashed), the timeline’s tests serve as the monitor. The official grader sees only the final patch.

85 **Attribution.** A regression is attributed to a harness check if a failing check and the broken held-out
86 test both exercise a file the agent changed at that observation, using a per-instance coverage map
87 built at the base commit. We report attributed detection, co-occurrence (any harness failure while the
88 regression was open, an over-count), and unknown separately.

89 **Execution environment.** The agent’s tools and the harness’s verification checks execute inside the
90 same pinned container as the replay, with file changes synchronised between the container and the
91 host tree that the timeline records. Section A.2 describes why this was not initially the case and what
92 it cost.

93 **Validation.** Five gates run before any paid experiment: a negative control, a positive control
94 with injected regressions including recovered ones, a flake screen, an unknown-rate ceiling, and
95 a baseline liveness check that the probes return passes. A golden check drives the benchmark’s
96 gold patch through the agent’s real tool path and requires the grader to return "resolved" (and a null
97 run "unresolved"); it passed on the three repository families with the most distinctive runners. A
98 re-scoring control re-measures an archived run’s committed timeline under a changed instrument
99 with no model calls; run twice, it reproduced the archived episode counts exactly.

100 4 Experimental setup

101 **Benchmark and slice.** SWE-bench Verified, 500 instances. A 40-instance development slice, drawn
102 by stratified sampling and fixed before any measurement, was used for all experiments here; it is
103 excluded from the confirmatory frame that a later study would use.

104 **Harness.** A planner decomposes the task into steps, each with a verification command; a worker
105 executes each step with three tools (read file, write file, run shell); a monitor runs the verification. On
106 failure, the recovery policy acts. Three policies were compared: **rollback** (reset to the last verified
107 checkpoint and retry with feedback), **repair-in-place** (retry from the failed tree), and **no recovery**
108 (keep the failed step’s tree and continue). Each run had a \$4 work-cost cap; budget exhaustion is
109 scored as failure.

110 **Models.** Two frontier stacks were run under identical harness, policy, instances, and caps: Claude
111 (Opus 4.7 planner, Sonnet 4.6 worker) and GPT-5.6 (sol planner, terra worker). Model identifiers are
112 recorded in every run’s manifest.

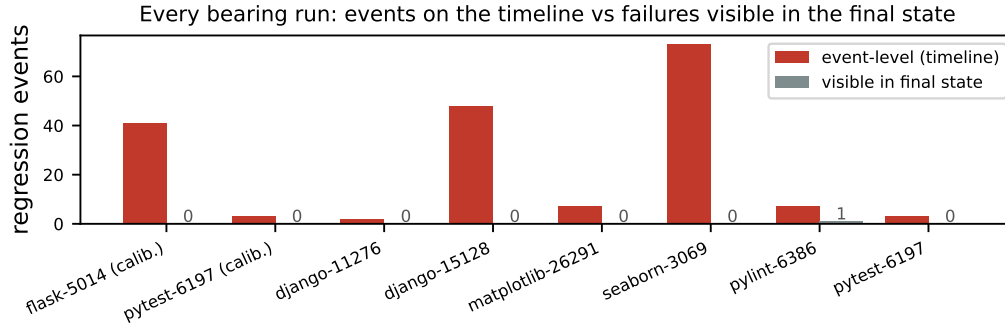


Figure 2: Every run that produced at least one regression event, in both GPT-5.6 sweeps. Red bars are events recorded on the timeline; grey bars are held-out test failures visible in the graded final patch.

Outcomes. Resolve is the official grader’s verdict on the final patch. Event counts are reported at three levels because the distribution is heavily overdispersed: distinct incidents (observations at which at least one test broke), declared events (one per test function and onset, parametrised variants collapsed), and bearing runs (runs with at least one event). Final-state contamination is the number of held-out tests failing in the graded final patch, net of baseline-dead tests.

Pre-declaration. The event unit, the gates, and the contrast were declared before the relevant data existed; the contrast’s analysis (exact paired sign tests on final-state contamination as primary and incident exposure as co-primary) was committed before either comparison arm finished. One amendment, making the gates conditional on the model stack, preceded unblinding. Appendix B gives the wording.

5 Results

5.1 Resolve and cost

Under rollback, the Claude stack resolved 13 of 40 instances (32.5%, exact 95% CI 18.6% to 49.1%) at \$34.24; the GPT-5.6 stack resolved 13 of 35 graded instances (37.1%) at \$8.14. The GPT-5.6 sweep’s circuit breaker stopped three cells after six consecutive zero-progress failures in one repository family, and two cells failed on a file-synchronisation defect; both are counted as attempted and not resolved.

5.2 The final state hides almost all regression activity

Figure 2 shows every run that produced a regression event, under the rollback policy with the GPT-5.6 stack. The timeline recorded 184 declared events across eight runs. The final patch exposed one. Every other event was created and then removed by the harness’s rollback before the run ended, so the graded patch was clean. The same measurement made from the final patch, which is how prior work measures regressions [8], would report one event in eight runs.

Figure 3 shows a single run in detail. The grader marked it resolved with all graded tests passing. Its timeline contains three regressions of the same test function at observations 3, 5, and 7, each repaired by rollback. Two of the three were not detected by any harness check while they were open; the harness noticed nothing and the rollback that repaired them was triggered by an unrelated failure.

Across the full GPT-5.6 sweep, 48 of 162 raw episodes had no co-occurring harness failure of any kind.

5.3 Regression frequency depends on the model stack, not the benchmark

On the same 40 instances, under the same harness and rollback policy, the Claude stack produced no regression events in 40 runs (227 observations, all replayed). The GPT-5.6 stack produced 140

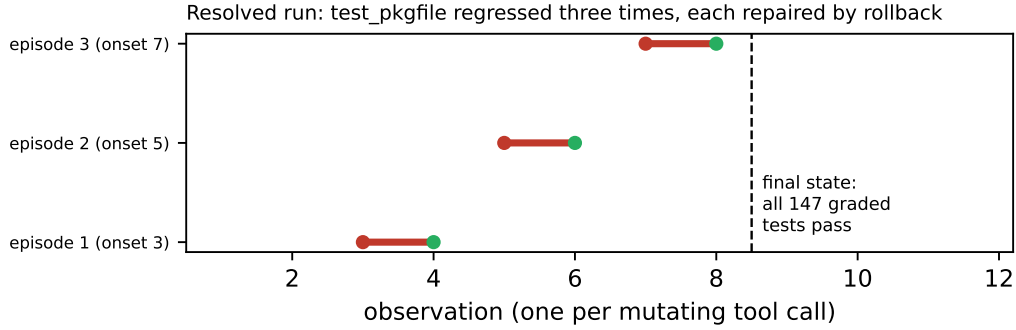


Figure 3: One run that the official grader marked resolved, with all 147 graded tests passing. The timeline shows three regressions of the same test function, each repaired by rollback two observations later.

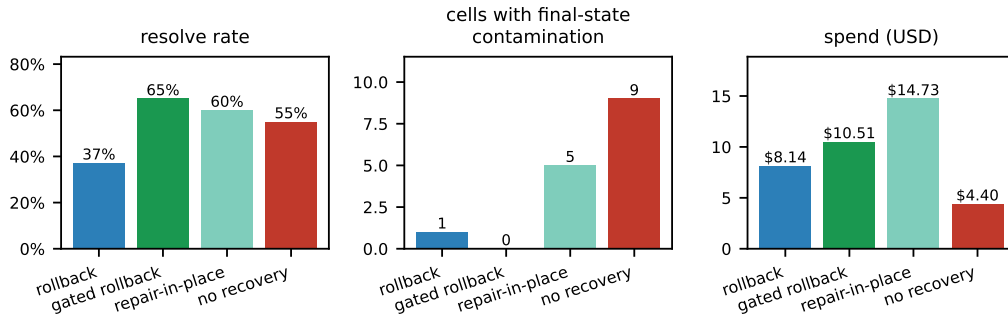


Figure 4: The four recovery arms under the GPT-5.6 stack on the same 40 instances. Resolve rate is over graded cells; contamination counts cells whose final patch fails a previously-passing test, net of baseline-dead tests.

145 declared events in 11 incidents across 6 of 37 attempted runs (bearing fraction 16.2%, exact CI 6.2%
 146 to 32.0%). A one-sided Fisher exact test on the bearing fractions gives $p = 0.010$. Observation density
 147 differs by a factor of 1.4 between the stacks, which does not account for a difference of 140 to 0.

148 The Claude zero is a measured zero. The same stack’s earlier two-instance canary run did produce
 149 three events, and re-scoring that run’s archived timeline under the instrument used for the 40-run
 150 sweep reproduces them exactly, so detection did not fail; these 40 runs did not break adjacent
 151 behaviour.

152 We had first read the low event rate as a property of the benchmark, and a pre-declared rule directed
 153 a switch to one with a larger held-out set; the second stack falsified that on the same instances.
 154 Regression frequency is a property of the agent regime, and any gate on it must be declared per
 155 regime.

156 5.4 Recovery policy: rollback keeps the final tree clean

157 Figure 4 summarises the pre-declared contrast. The primary endpoint, final-state contamination, is
 158 paired by instance: no recovery was worse than rollback on 9 instances, better on 1, and tied on 25
 159 (exact sign test, $p = 0.022$). Repair-in-place was worse on 5, better on 1, and tied on 29 ($p = 0.22$).
 160 The co-primary endpoint, incident exposure, did not differ between policies ($p = 0.45$): regressions
 161 occur under every policy at similar rates, and the policy determines whether they persist.

162 One no-recovery run illustrates the mechanism: its final tree begins with the literal string `$(cat`
 163 `django/db/models/expressions.py)`, a shell idiom pasted into a file write, and none of the

164 instance’s 137 graded tests ran. Under rollback the same model’s failed attempt on the same instance
165 was reset, and the final patch passed all 137.

166 **Disclosure.** The first unblinding gave $p = 0.375$ for the primary endpoint: seven cells (five no-
167 recovery, two repair-in-place) had been dropped as ungradable because their final trees made the suite
168 uncollectable and the grader returned no result. The official rule scores such a patch as failing every
169 test, so the most contaminated final states had been removed from the endpoint they bore on most.
170 The grader was corrected to distinguish an environment failure (no results with or without the patch)
171 from a patch that kills the suite, and the seven archived trees were re-graded without re-running any
172 agent.

173 5.5 Rollback loses resolution to verifier precision

174 Rollback resolved fewer tasks than either alternative (Figure 4, left): 37% against 60% for repair-in-
175 place and 55% for no recovery. Paired by instance, rollback won one discordant pair and lost nine
176 against repair-in-place (McNemar exact $p = 0.022$), and won three and lost nine against no recovery
177 ($p = 0.15$).

178 The loss is attributable to the verifier. Of the 18 rollback runs that failed, 15 failed at the first
179 step, and 9 of the 18 instances were resolved under a policy that kept the rejected work. In those
180 cases the monitor’s check, a shell command written by the planner, rejected a patch that the official
181 grader accepted, and rollback discarded it. Rollback’s advantage on the primary endpoint and its
182 disadvantage on resolution therefore have the same cause: it acts on the verifier’s verdict, and the
183 verifier’s precision sets the exchange rate between a clean final tree and a solved task.

184 This suggests replacing the planner-written check with the repository’s own tests, selected as the
185 instrument selects its oracle, and triggering rollback only on a genuine regression. Section 5.6
186 evaluates it.

187 5.6 Regression-gated rollback removes the tradeoff

188 A fourth arm replaces the monitor’s planner-written check with the repository’s previously-passing
189 tests, run in the agent’s container at the start of the run and after every attempt; a step is rejected
190 only if a test that passed at the start fails now. Everything else (policy, model, instances, caps) is
191 unchanged. It resolved **26 of 40 instances (65.0%)**, the highest of any arm, and left **no contaminated**
192 **final tree** (Figure 4). Paired against plain rollback on the 35 shared instances, it resolved 10 that
193 plain rollback did not and lost 1 (McNemar exact $p = 0.012$). Regression exposure was unchanged
194 (70 declared events in 5 bearing runs; co-primary $p = 0.73$): the gate does not stop the model from
195 breaking adjacent behaviour, it catches the breakage with the tests that define it and resets. The three
196 gated runs that failed each failed on a genuine regression the gate refused (8, 2, and every test), and
197 the repository family that had tripped the plain arm’s circuit breaker completed. The suite runs cost
198 \$2.37 over plain rollback.

199 6 Discussion

200 **Implications for agent-OS design.** If checkpointing and recovery become shared primitives, timeline
201 observability must be one too: a recovery primitive without an observation primitive makes runs look
202 cleaner while hiding the activity that distinguishes policies. The 184-to-1 ratio is what any final-state
203 grader will report for any harness that repairs its own mistakes.

204 **On the measurement defects.** Building the instrument produced 28 defects (Appendix A); 23
205 produced a plausible number rather than an error, and 27 were present while a passing test suite
206 watched. They fall into four mechanisms, each with public instances in benchmark infrastructure
207 (Appendix A.3). Two cheap rules prevented most of them: type infrastructure failure as a missing
208 observation, never as a verdict; and require the instrument to prove it is alive, not merely that it
209 reported nothing.

7 Limitations

All measurements come from a 40-instance development slice, one sweep per arm and per stack; run-to-run variability is unmeasured, and the confirmatory design specifies seeded replications on held-out instances. The cross-stack comparison cannot separate a model from its provider adapter. The held-out tests observe roughly the gold test patch’s blast radius, so event counts are lower bounds. Confidence intervals treat the fixed slice as the population and ignore repository clustering (46% one repository). The harness is our own; the instrument has not yet been applied to a public scaffold.

8 Conclusion

Final-state evaluation of coding agents is blind to regressions that the agent’s own process repairs, and in our measurements that is nearly all of them. Measuring the timeline is feasible, validates against controls, and separates recovery policies that the final patch cannot distinguish. It also exposes the cost of recovery, and pays it back: a rollback triggered by the repository’s own tests kept every final tree clean while resolving more tasks than any other policy.

References

- [1] C. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? ICLR 2024. arXiv:2310.06770.
- [2] OpenAI. Introducing SWE-bench Verified. 2024.
- [3] L. Zhang et al. SWE-bench Goes Live! arXiv:2505.23419, 2025.
- [4] X. Deng et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? arXiv:2509.16941, 2025.
- [5] I. Badertdinov et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. arXiv:2505.20411, 2025.
- [6] S. Liang et al. The SWE-Bench Illusion: When state-of-the-art LLMs remember instead of reason. arXiv:2506.12286, 2025.
- [7] H. Yu et al. UTBoost: Rigorous evaluation of coding agents on SWE-Bench. ACL 2025. arXiv:2506.09289.
- [8] TDAD: Regressions introduced by coding agents on SWE-bench Verified. arXiv:2603.17973, 2026.
- [9] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. NeurIPS 2024. arXiv:2405.15793.
- [10] X. Wang et al. OpenHands: An open platform for AI software developers as generalist agents. ICLR 2025. arXiv:2407.16741.
- [11] C. Xia et al. Agentless: Demystifying LLM-based software engineering agents. arXiv:2407.01489, 2024.
- [12] SWE-agent team. mini-swe-agent. Software, 2025.
- [13] X. Wang et al. Executable code actions elicit better LLM agents. ICML 2024. arXiv:2402.01030.
- [14] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. ICLR 2023. arXiv:2210.03629.
- [15] N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023. arXiv:2303.11366.
- [16] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. NeurIPS 2023. arXiv:2303.17651.

- [17] ReflexGrad: Within-episode failure recovery in LLM agents via progress-gated dual-process routing. arXiv:2511.14584, 2025.
- [18] REPOT: Recoverable program-of-thought via checkpoint repair. arXiv:2605.30052, 2026.
- [19] From agent traces to trust: Evidence tracing and execution provenance in LLM agents. arXiv:2606.04990, 2026.
- [20] K. Mei et al. AIOS: LLM agent operating system. arXiv:2403.16971, 2024.
- [21] Holistic Agent Leaderboard: The missing infrastructure for AI agent evaluation. arXiv:2510.11977, 2025.
- [22] What resolve rate hides: Trajectory structure diagnostics for coding agents. arXiv:2607.06184, 2026.
- [23] SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. arXiv:2512.18470, 2025.
- [24] Agentic rubrics as contextual verifiers for SWE agents. arXiv:2601.04171, 2026.
- [25] SWE-Shepherd: Advancing PRMs for reinforcing code agents. arXiv:2604.10493, 2026.
- [26] Patch overfitting in program repair: A survey. 2024.
- [27] Y. Wang et al. When automated program repair meets regression testing: An extensive study on two million patches. ACM TOSEM, 2024. arXiv:2105.07311.
- [28] Patch correctness assessment: A survey. ACM TOSEM, 2024. doi:10.1145/3702972.
- [29] H. Ye et al. Automated patch assessment for program repair at scale. Empirical Software Engineering, 2021.
- [30] SWE-bench issue #601: Test result hijacking via stdout forging in the evaluation harness. github.com/SWE-bench/SWE-bench, 2025.
- [31] OpenHands issues #4235 and #7044: Environment errors in SWE-bench instance evaluation. github.com/OpenHands/OpenHands.
- [32] J. Yang et al. SWE-smith: Scaling data for software engineering agents. arXiv:2504.21798, 2025.
- [33] J. Pan et al. Training software engineering agents and verifiers with SWE-Gym. arXiv:2412.21139, 2024.
- [34] Context as a tool: Context management for long-horizon SWE agents. arXiv:2512.22087, 2025.

A The measurement defect catalogue

A.1 The catalogue by mechanism

Each row is a defect found while building the instrument. **S**: it produced a plausible number rather than an error. **G**: it was present while the test suite passed (A4 is the suite). **H**: it was findable only on a clean host. Class A rows depend on the machine the code runs on; class B rows render an infrastructure failure as a measurement; class C rows measure a state where an event was defined; class D rows consume a producer’s output without checking the producer.

#	Defect	Consequence	S	G	H
A1	Shadow commits inherited the machine's git identity	timeline silently empty on any clean machine	✓	✓	✓
A2	Gate probe invoked bare <code>python</code>	every probe exit 127 on a clean host	✗	✓	✓
A3	Unpinned SDK resolved a different major version	two machines ran different code	✗	✓	✓
A4	Test fixtures verified with bare <code>pytest</code> from PATH	15 tests fail on a clean host as harness defects	✗	—	✓
A5	A benchmark scorer invoked bare <code>python</code>	score 0.0 from a missing interpreter	✓	✓	✓
A6	Agent executed on the host; measurement in the container	26 of 28 zero-step runs; see A.2	✓	✓	✓
B1	Output marker used shell :	a 13/13 run scored as 13 errors	✓	✓	
B2	Results on stderr, markers on stdout	one framework's results outside the parsed slice	✓	✓	
B3	Probe diffed against a deleted commit	every graded test a hole	✓	✓	
B4	Interrupted mirror clone accepted with zero commits	later instances fail far from the cause	✓	✓	
B5	Container workdir unvalidated	every exec exit 127	✓	✓	
B6	Isolation removed loopback, not only the internet	23.5% of the oracle dead	✓	✓	
288 B7	Provider cache served removed containers	later cells replay against nothing and score clean	✓	✓	
B8	Tree reset reverted image build-time edits	one repository family's oracle dead	✓	✓	
C1	Bisection assumed monotone verdicts	recovered regressions invisible	✓	✓	
C2	Declared event unit not implemented	rate off by up to $2.7\times$	✓	✓	
C3	Rollbacks produced no observation	recoveries invisible to the timeline	✓	✓	
C4	"Persists past the step boundary" undefined	one reading excludes the events of interest	✓	✓	
D1	Audit field never populated	0 tool errors reported, always	✓	✓	
D2	Malformed planner output crashed the run	33% of runs lost as task failures	✗	✓	
D3	Dependency install counted as agent work	attribution vacuous	✓	✓	
D4	Join took the first observation, not the last	failures pinned to the wrong tree	✓	✓	
D5	Executed config rebuilt from a name	manifest described a different run	✓	✓	
D6	Git handles never released	sweeps die after ~400 cells	✓	✓	
D7	Console re-walked the tree per run	presented as a dead server	✗	✓	
D8	Detection events lacked the ids attribution joined on	attributed detection always unknown	✓	✓	
D9	Absent coverage rendered as measured silence	unmeasured episodes reported as silent	✓	✓	
D10	Output-capped turns executed	half-written file; a 78-event storm	✓	✓	

289 Totals: 23 of 28 silent; 27 of 28 present under a passing suite; 6 of 28 findable only on a clean host.
290 Four further defects found after this census closed are described where they arose (Section 5.4 and
291 the repository log).

292 A.2 The defect that invalidated a conclusion

293 After nineteen fixes, every validation gate passed, the re-scoring control reproduced archived episode
294 counts exactly, and three pilots (80 runs) measured an event rate far below the pre-declared gate.
295 We drafted the conclusion the rule directed: the benchmark offered too little opportunity, so switch
296 benchmarks. The conclusion was wrong. The harness ran the agent on the host in a bare source
297 checkout, while every probe and every gate ran inside the pinned container. On the host, `import`
298 `matplotlib` in that checkout succeeds by importing the uncompiled source tree as a namespace
299 package, and everything downstream fails in ways indistinguishable from an incompetent agent.
300 Twenty-six of 28 zero-step runs traced to this. The gates had validated the measurement path and
301 never the agent's execution path. The fix was to route the agent's tools and checks through the same
302 container as the replay, and to add a per-cell environment parity check that runs before any model
303 call.

304 A.3 Mechanisms in public infrastructure

305 Class A: OpenHands issues #4235 and #7044 [31], environment construction failures against SWE-
306 bench images. Class B: UTBoost's finding that the held-out oracle is insufficient at leaderboard scale
307 [7], and, on SWE-bench Live, baseline tests that fail in the unmodified image because the calendar
308 passed a deprecation date encoded in the instance. Class C: final-state regression measurement [8].
309 Class D: the SWE-bench grader accepting forged test output on stdout [30].

310 **B Pre-declaration and analysis**

311 The event unit is one (test function, onset observation) pair with parametrised variants collapsed. The
312 gates for proceeding on a substrate were an event rate of at least 0.30 per run and a bearing fraction
313 of at least 25%; after the cross-stack result they were re-declared per (substrate, model stack) before
314 the contrast was unblinded. The contrast's primary endpoint is final-state contamination, paired by
315 instance, exact two-sided sign test; the co-primary is incident exposure. The analysis script was
316 committed before either comparison arm finished. Budget exhaustion is scored as failure. Instances
317 whose baseline oracle records failures in the unmodified image are excluded from resolve-rate
318 comparability claims, and their dead tests are typed as missing observations for event counting.